

Automating modern code review processes with code similarity measurement

Yusuf Kartal ^{a,*}, E. Kaan Akdeniz ^b, Kemal Özkan ^a

^a Computer Engineering, Eskisehir Osmangazi University, Eskisehir, Türkiye

^b Computer Engineering, Izmir Institute of Technology, Izmir, Türkiye

ARTICLE INFO

Keywords:

Modern code review
Vectorization
Code similarity
Information retrieval

ABSTRACT

Context: Modern code review is a critical component in software development processes, as it ensures security, detects errors early and improves code quality. However, manual reviews can be time-consuming and unreliable. Automated code review can address these issues. Although deep-learning methods have been used to recommend code review comments, they are expensive to train and employ. Instead, information retrieval (IR)-based methods for automatic code review are showing promising results in efficiency, effectiveness, and flexibility.

Objective: Our main objective is to determine the optimal combination of the vectorization method and similarity to measure what gives the best results in an automatic code review, thereby improving the performance of IR-based methods.

Method: Specifically, we investigate different vectorization methods (Word2Vec, Doc2Vec, Code2Vec, and Transformer) that differ from previous research (TF-IDF and Bag-of-Words), and similarity measures (Cosine, Euclidean, and Manhattan) to capture the semantic similarities between code texts. We evaluate the performance of these methods using standard metrics, such as Blue, Meteor, and Rouge-L, and include the run-time of the models in our results.

Results: Our results demonstrate that the Transformer model outperforms the state-of-the-art method in all standard metrics and similarity measurements, achieving a 19.1% improvement in providing exact matches and a 6.2% improvement in recommending reviews closer to human reviews.

Conclusion: Our findings suggest that the Transformer model is a highly effective and efficient approach for recommending code review comments that closely resemble those written by humans, providing valuable insight for developing more efficient and effective automated code review systems.

1. Introduction

Depending on the technological changes experienced, developers' expectations of software development processes have changed. Studies on the quality of the developed software, the expected performance, and the approaches that can be realized have quickly become popular. With the progress made, time management has become a priority in large-scale projects [1]. Solving coding errors that developers encounter in a short time is essential in providing time management [2]. Although simple mistakes made by developers during the development phase may seem insignificant, they can lead to serious security problems in the later stages of the projects [3]. While focusing on software development processes in previous studies, new approaches where errors are noticed more quickly, and quality features such as compliance, merging, readability, and complexity in codes, gain importance [4]. Therefore, it is necessary to find an accurate solution

to bugs encountered by developers quicker in order to ensure that efficient time management in the projects is carried out. Bugs that are not noticed early on may occur during the testing phase of the software developed, or after it is delivered to the user. Solving bugs that developers encounter in a short time is essential in providing time management.

Operations to improve code quality are performed in many parts of the software development process: the rules and instructions to be followed before development; the documents to be read; the auxiliary tools available to the editors; the tests written during the development process; the code reviews; and running certain automated control tools after the completion of the development. These are the primary known operations to improve code quality [5]. One part of this process that requires the most human effort is the code review process [6,7]. Code review activities have become essential in the detection of potential errors early on, to improve coding approaches, and to disseminate

* Corresponding author.

E-mail address: ykartal@ogu.edu.tr (Y. Kartal).

<https://doi.org/10.1016/j.infsof.2024.107490>

Received 11 May 2023; Received in revised form 12 May 2024; Accepted 13 May 2024

Available online 17 May 2024

0950-5849/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

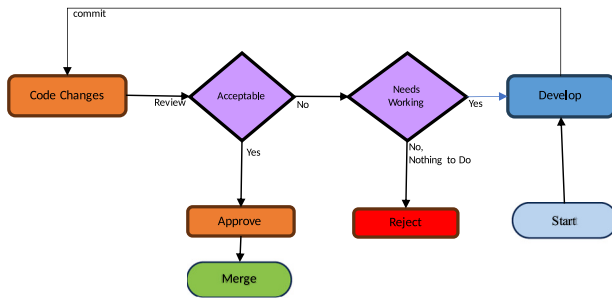


Fig. 1. Modern code review process.

information among software developers [8]. With this more efficient use of hosting and collaboration tools, code review processes that previously took the form of meetings on a specific piece of code have evolved into a process in which one or more developers review all added, updated, and deleted code lines, the review is forwarded, and they are included in the main branch with team approval [9]. During the Modern Code Review process, team members manually review the committed code, and necessary comments are made on any bug-detected source code piece. Developers may add additional comments, reply to comments, or fix the defective source code and notify reviewers to check redeveloped parts. This process continues until the code is approved by everyone (Fig. 1) [10]. Therefore, potentially buggy codes may be detected at an early stage, a quality coding standard is achieved, and less time is spent on detecting defects [11].

Recent studies have shown that code that goes through ‘Modern Code Review (MCR)’ is less likely to contain bugs and contributes to the emergence of high-quality code with high readability and maintainability [12].

Along with MCR activities, various methods and techniques for making predictions for newly added code lines, based on comments specific to code lines during a code review by senior software developers, have begun to be developed. To improve estimation accuracy, models with various working logic have been developed [12–16]. Let us suppose that most of the common mistakes made in general have been detected. In such a case, senior software developers can focus on the parts that require more detailed domain knowledge, rather than wasting time reviewing simple errors.

Recent studies [13–15] have generally been carried out to increase the success of deep learning-based generation models. In these studies, we can list some tasks as follows;

- The generation of the code to be proposed for the examined code.
- The generation of the code to be proposed using the examined code and comments.
- The generation of direct comments on the examined code.

In our study, we focus on generating direct comments on the examined code. Looking at previous studies, it can be seen that the studies on code review still cannot achieve the expected results. In addition to this, different methods applied on different datasets sometimes have a number of problems, such as high training costs or high prediction times. On the other hand, it has been shown that a mechanism that would work as an Information Retrieval (IR)-based recommendation system could be more successful [12]. The aforementioned study created a recommendation system using the bag-of-words and the cosine distance methods. The results were evaluated by calculating the BLEU score between the actual and suggested comments.

In this study, we aim to create many combinations using the methods mentioned in the third section to reach a comprehensive output. We therefore focus on reaching a conclusion with regard to increasing the success of the previously proposed Information-Retrieval-based (IR-based) code review suggestion system [12]. On the other hand, the

system we propose aims to automatically detect repetitive and simple errors in the code reviews of software teams with a specific review history and standards rather than trying to produce comments suitable for every code change it sees. Such a system would reduce certain problems that occur after a developer has changed in software teams, such as failing to follow the standards and decreasing the code quality. For example, a medium-sized software company has particular standards, and a specific software culture has been formed regardless of the duration of the project it has developed. When the recommendation system is based on the code review history here, both customized and more accurate code review recommendations will be made. In addition to customization, in cases where generalization is desired, the desired results can be easily obtained by expanding the database. We assume that the better we can measure the lexical similarity of code pieces, the more accurate our recommendations will be. The idea is that if similar reviews can be conducted with similar code changes that are not so obvious, the number of best candidates can be increased for recommendation to the reviewer. Furthermore, studies could be carried out in the future to determine which of the best candidates is better than the others.

Our contributions are as follows:

- We conduct an evaluation of various vectorization and vector distance measurement methods to assess their effectiveness and efficiency in measuring code similarity within the context of code review recommendations. We provide a comprehensive perspective on the performance of these methods designed for natural language texts when applied to source code. Our findings not only offer insight for proposing more successful methods in code review proposals, but also provide guidance for future studies in this area.
- We evaluate the performance of neural network-based vectorization methods on code texts and compare them with classical methods used in previous studies [12,14]. Specifically, we investigate the effectiveness of Transformers, a popular neural network architecture in natural language, on code texts. Our results indicate that the Transformer performs well on code texts, surpassing the success rates achieved by prior studies.
- We employ a 5-fold cross-validation approach throughout all experiments to enhance the robustness of our findings and to demonstrate their generalizability, thereby ensuring that our results remain consistent across diverse datasets [17].

This paper aims to convey the evaluation results obtained from the methods we applied to the researchers who want to work in the field of source code review. It discusses the studies made in the source code review field in Section 2, and Section 3 details the method we applied for automatic source code review. Section 4 includes the experimental results and their explanations, and Section 5 discusses the study’s conclusion.

2. Related work

With the large-scale increase in the number of software companies and their increasing complexity, traditional manual methods fail to deal with numerous problems such as security, performance, quality, and so on. Therefore, identifying potential bugs or vulnerability issues in software is an essential area of research for researchers in the field of the software development process. Recently, many studies have been conducted in the software development process field [12,14,15,18–20]. We examined topic-related studies in detail. There are many software development topics, with most seeming similar, but each case needs a separate study. Our focus is on papers that studied MCR recommendations.

Considering that the similarity with previous codes will be taken as a basis to suggest comments in the review activities, it can be seen that code similarity is studied on issues such as code clone detection, error

detection, and plagiarism detection. Ragkhitwetsagul proposes a system for online code reuse discovery, large-scale code clone detection, the use of case finding, software plagiarism detection, and software licensing conflicts detection, incorporating not only text similarity, but also syntactic and structural similarities [21].

Gupta and Sanderson have created a deep learning-based system that learns how to make suggestions for similar problems detected in source codes with their study called DeepCodeReview. They conducted a survey to measure the success of this system [22].

Tufano et al. aimed to design a system that would make suggestions to both the code developer and the reviewer with two different deep-learning architectures. These architectures are designed to produce correct code-to-code and code-to-comment matches. Although they achieved remarkable results in the systems they developed, they concluded their studies by stating that more work needs to be done in order for them to be fully functional [13]. After this, by changing their approach, they proposed a new system using ‘Text-To-Text Transformers’ in a second study. They argued that the proposed system was more efficient than the approaches implemented with deep learning [13]. In addition to their previous studies, Tufano et al. stated that they had expanded the dataset significantly and had revealed a more successful model by preprocessing and fine-tuning the dataset. They tested with both the pre-trained and non-pre-trained models and mentioned that the pre-training of the model was quite effective [16].

Siow et al. aimed to outperform static analysis or deep learning-based approaches in their work, which is based on code changes and related reviews. Therefore, they stated that they had achieved more successful results by adding text embeddings to the deep learning model they had used [14].

Hong and Tantithamthavorn propose a system named Comment-Finder. It is a retrieval-based approach based on vectorizing code blocks using the bag-of-words method and measuring the similarity of code blocks by measuring the closeness of these vectors using the cosine similarity method. In addition, it is 32% better than prior work for perfect prediction when recommending the best-1 code review comment. It not only gives better correctness but is also forty-nine times faster than previous work [12].

Li et al. propose a pre-trained deep-learning model named CodeReviewer. With this approach, at the first stage of the code change, whether the change is of high quality, and therefore ready for review, it is given in binary (0, 1). In the second step, the CodeReviewer focuses on suggesting the best matches found for a code change with an approach that is based on code diff and comments and leaving the appropriate one to the developer’s choice. In the third stage, they developed a system to show the developer how to reorganize the code according to the comments made on the code. According to their findings, their model outperforms the previous state-of-the-art deep-learning-based approaches [23].

Li et al. designed a system called AUGER that generates code review comments using the Text-To-Text T5 model [15]. It achieved a 37.38% increase in the ROUGE-L score compared to the baseline [16]. The elements that increased the performance of this study were the preprocessing phase of the data, review line tagging, which is labeling the part of the code under review, cross pre-training to make the relationship between the code and the review more explicit, and the masking method. This study shows the importance of data adaptation in generating code review comments.

Shuvo et al. created a new data set from GitHub. The dataset contains approximately 56k code changes and comment pairs. They kept 70% of the instances for the corpus and the remaining as the query. They reported that they achieved more successful results with their IR-based model than the methods they compared in their study [2].

From another perspective, Tufano et al. in their study in 2024, aimed to reveal the strengths and weaknesses of studies on code review. They performed manual analyses. Thus, they focused on whether

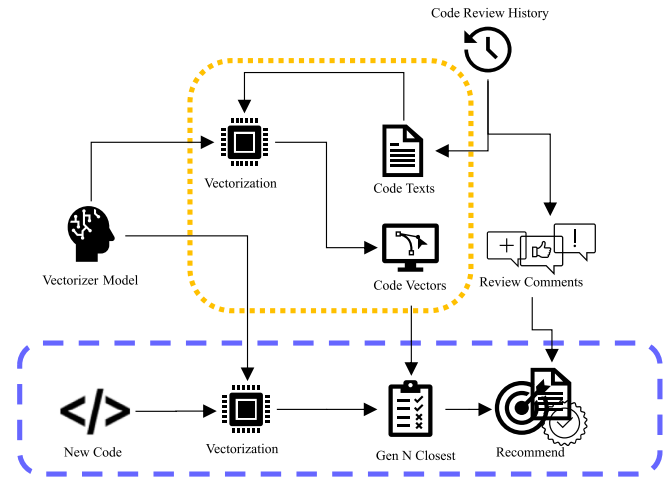


Fig. 2. Code review recommendation steps.

the predictions made using three different techniques were correct or incorrect [24].

Lu et al. stated that Large Language Models (LLMs) have not been studied in detail for automating code review tasks. They reported that their LLaMA-Reviewer achieved high performance with very few features by significantly reducing resource usage [25].

A study by Sarker et al. stated that instead of producing or recommending code comments, they determined the best combination among many algorithms to clean toxic code comments. They concluded that their proposed system’s accuracy and F1 scores were quite high [26].

In addition to automating code review processes, many studies have been conducted on other sub-tasks for source codes. The most well-known of these is CodeBERT [27], a Transformer-based system. The main purpose of this work is to represent the semantic relationship between natural language and code. It has been used in tasks where code and natural language are used together, such as in the code documentation generation. It is an important milestone in making models, such as BERT, for natural language work in code text. In the CodeXGlue study [28], a benchmark dataset was created for subtasks on code understanding and generation. In this study, the CodeBERT model was used for the code clone detection task related to our task. Furthermore, different models were finetuned for many other subtasks, such as code completion, code summarization, and others, with the results being shared.

There are different works on the vectorial representations of codes, which we also focus on. The literature review by Chen and Monperrus [29] provides a comprehensive survey of the work on code embeddings. In addition, before CodeBERT, Kanade et al. developed a contextual code representation model called CuBERT [30] by pretraining the BERT model with code texts. In addition, in UniXCoder [31], which we also use as a Transformer model, Guo et al. used a multi-layer Transformer architecture for the representation of code texts and tokenized the code texts by way of the abstract syntax tree (AST) parsing method. Closely related to this work, Niu et al. developed a sequence-to-sequence model for code representation called SPT-Code [19].

3. Materials and methods

The studies conducted using different methods were examined in detail to achieve our aim, and the repeatable tests included in these studies were performed again. We propose the Transformer for vectorization of source codes to be used with similarity functions. As in many studies, channels such as GitHub could be used to create our dataset. However, the datasets in the literature were preferred to make comparisons with other methods more objective. Information about the

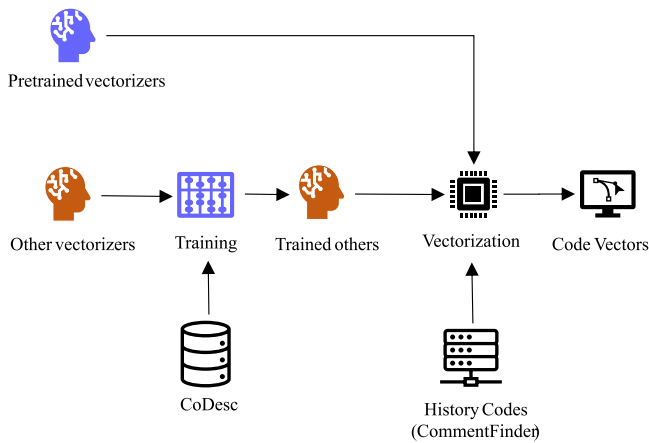


Fig. 3. Vectorization.

datasets is given in the following sections. In addition to the datasets used, the subunits of the constructed method are given in subtitles with their intended use. The overall flow of the system is shown in Fig. 2 from start to finish. The one-time flow for the system is shown by the dotted frame. The continuous system flow is shown by the dashed frame. Details of the generation of code vectors from the code review history (dotted frame) in the experimental phase are shown in Fig. 3. More details about the system flow are given in the Experiment Design section.

3.1. Datasets

This section gives information regarding the datasets used in training the models, evaluating the proposed system, and how they are used.

3.1.1. Codesc

We trained the vectorizer models we used in the experiments with a large code dataset to ensure they were flexible and adaptable to data changes. Vectorizers other than Unixcoder are trained with CoDesc [32] data, as shown in Fig. 3. This dataset contains 4.2 million Java source codes. We chose this dataset as the corpus of the vectorizer models. As can be seen in the studies [33–35], embedding performances increase proportionally as the corpus size grows. As a result, vectorizer models trained with data of this size can more appropriately represent vectorial representations of source codes. At the same time, the aim is to eliminate fine-tuning and retraining costs that will arise in the case of new data by creating a general model.

3.1.2. CommentFinder

This dataset, containing approximately 150,000 code-review pairs, created in the previous work [12] to represent the code review history shown in the system flow (Fig. 2) to perform the experiments and identify the model that performs the best code review according to the metrics we measure is used. For experiments, code texts in this dataset are vectorized with all vectorizer models, as shown in Fig. 3. After this, the dataset is divided into five parts for 5-fold cross-validation, and each 30,000 pairs part is taken as the code sent for review, shown as the ‘New Code’ part in the system workflow (Fig. 2). At the same time, 120,000 represent that iteration’s existing code review history. In the previous study [12], a test/history selection of 15,000/135,000 was made with a fixed split, and no cross-validation was applied. The test data in each iteration in our study is twice as big as the test data of the mentioned study. In addition, the aim is to obtain more verifiable results from the results obtained from fixed test data with cross-validation. After cross-validation, all iterations are averaged based on determined metrics.

3.2. Experiment design

The experimental procedures were devised in accordance with the system flow diagram depicted in Fig. 2. The system, as designed, incorporates a one-time flow that operates at the outset to generate a database comprising pairs of code vectors and corresponding review comments. This database is created from the historical code review data of the software team that will be utilizing the system. Subsequently, the continuous flow of the system runs for every new code segment submitted for review.

In a hypothetical scenario, a software development team has been working on a project named ‘X’ for a period of two years, and their code review history spans the same duration. In this scenario, the automatic code review system vectorizes all the code present in the history and stores these vectors alongside their corresponding review comments in a database. Upon submitting a new code segment for review, the system vectorizes the new or modified code and computes its similarity with the vectors in the code history database. The automated system provides review suggestions by identifying the most similar code vectors from the database. The number of suggestions provided depends on the selection of candidates.

For our experimental setup, we utilize the CF dataset to represent the code history and newly added code, as detailed in the Datasets section. With this system design, we help identify common bugs and assist software teams in upholding their standards.

A similarity measurement process is carried out between the code submitted for review and the codes in the review history, as shown in Fig. 2. The comments made to the most similar codes are presented as review suggestions. Unlike the code review comment generation tasks, the aim is to automate this process with the IR methods suggested in the previous study [12] and to create a system that gets better results than this suggested method. Furthermore, in CommentFinder (CF), the code texts were represented only by the BoW (CountVectorizer) method, and different experiments were carried out with different vector distance methods. In addition, we aim to measure the contribution of different vector representations (embeddings) of codes to the system in this study. In addition, experiments in CF were performed with a fixed-selected test set. We tried to provide more provable data to show the reliability and applicability of the results by performing 5-fold cross-validation in our experiments. Although the same code pieces may have more than one review due to pull requests at different times, we assumed that each fold represents a different code history.

We describe the methods that were used in the experiments. Three methods are used in our research: text embedding methods for maintaining vectorial representations of source code; text similarity methods to calculate the similarity between real comments and recommended comments; and vector distance methods for calculating the distance between source code embeddings. Our purpose is to find the best combination (model) to get the best recommendations in terms of code review. In this article, we call ‘model’ the sum of all the components that make up the automatic code recommendation system. In other words, each combination is a “model”.

3.2.1. Text embeddings

The vectorizer model in our workflow, shown in Fig. 2, was determined by looking at the results of the comparison of six different models. These are the TF-IDF, Word2Vec, Doc2Vec, Code2Vec, UniX-Coder models, and the BoW recommended in CF. We will refer to the UniXCoder as the Transformer model for the remainder of the article. At first, we trained only with the dataset, which we defined as the code review history, as these models will have to fine-tune or retrain costs as the dataset grows. As a result, these models will not be required to reconstruct since they also contain a large amount of code and will not be affected by the size of the code review history.

3.2.2. Evaluation metrics for natural language generation (NLG)

We used evaluation metrics as used in natural language production, such as BLEU [36], METEOR [37], and ROUGE [38], to measure the performance of our proposed system. However, our main evaluation metric is BLEU to ensure that it can be compared with similar studies [12–16]. After selecting the k number of candidate codes that are most similar to the codes in the test dataset, which already has review comments, the closeness of the comments made to these codes with the original interpretation is measured with these metrics, and the aim is to represent the success of the system in this way. In previous studies, the results of review comment generation and information retrieval, as well as review automation methods were evaluated with the help of these metrics. Finally, the desired measurement is how close an automatic review recommendation can be to a manual one. In the experiments using all combinations, the average of each metric for all the test data and the number of exactly matched comments were calculated separately. In the case of more than one candidate, the candidate with the highest metric score was determined as that group's score. In addition to NLG metrics, Transformer or classical text similarity methods could be included in the experiments in future studies.

3.2.3. Vector distances

The basic logic in text similarity calculations is based on representing the text as vectors and then measuring the distances between these vectors using different methods. Our study aims to measure code similarities by applying the same logic on the source code text, and the different methods we mentioned in section i. are used and called vectorizers. In the 'Get N Closest' phase in Fig. 2, the vectorial representations of each code in the test set and the vectorial representations of each code in the code review history were compared using the Cosine, Euclidean, and Manhattan methods, starting with the idea that the closest code vectors were the most similar codes. Similar to the other metrics, the methods selected here were also tried with different combinations. The most successful vector distance measurement method was determined by looking at the average of the text evaluation metrics for the whole test set and the exact matches again.

3.2.4. Exact match/perfect prediction

One of the criteria for automated code review experiments is perfect predictions. These estimates were when the system suggested review, and the review comments to the test data code were exactly the same. It is expected to work, especially when repetitive review comments are automated in real code reviews. It is also expected to come into play in similar code pieces where these comments are made. Although the experiments are performed on code review data with different standards belonging to different software teams, perfect predictions show that similar mistakes can be made in different projects with different standards written by different teams.

3.2.5. Execution time

A previous study [12] reveals that efficiency is one of the significant differences between the IR-based code review systems and systems that generate code review comments. In our experiments, the execution times are calculated for around thirty thousand codes in the test data of different models. All of the codes in the code review history are vectorized with all vectorizer models, and each model's execution times are calculated with different metrics. The time elapsed during the vectorization phase does not need to be considered. This is because, in the real scenario, only one code that will come for review will be vectorized and, once approved, will be included in the review history. The vectorization time is negligibly short. On the other hand, the part where the system spends most of the time will be where vector similarities are measured. This metric represents the time spent on this part of the execution. The differences between the execution times of the models rather than the exact values show this representation more clearly.

3.2.6. Data preparation

All of the code texts in the CoDesc dataset were already tokenized. As a result, there is no need for any preprocessing. For the CF dataset, all of the code texts were tokenized with the javalang [39] library. In this library, tokenization is made using the Abstract Syntax Tree (AST). We are not conducting other preprocessing, such as removing punctuation or lowering all of the characters, because this type of information is important in the meaning of code. In addition, in both datasets, comments were already removed.

3.2.7. Manual investigation

To fill the gap caused by computer-assisted evaluations such as the BLEU score, manual analysis was performed on code-comment pairs to evaluate suggested comments and expected comments. For a manual review of the suggested comments, 300 records were randomly selected from the first 15,000 records with high cosine similarity but not an exact match and low BLEU score. When the test data is examined, it is seen that even though the cosine similarity is high, there are low BLEU scores between the suggested and expected comments. For this reason, we tried to include those with high cosine similarity but low BLEU scores in this evaluation. The selected 300 records were examined separately by Avkar Software Company employees located in the Eskişehir Technology Development Zone. Each entry was independently reviewed by 10 separate developers. All software developers have at least five years of Java software development experience, and three people have a master's degree in Computer Engineering. One of the examiners is a Mathematician, and the others are Computer Engineers.

Software developers were given four options to choose how appropriate the proposed interpretation was. These options are; a. *Suitable*, b. *Same meaning*, c. *Alternative Solution*, d. *Not suitable*

The first three options indicate situations in which the recommendations are valid. The other option indicates invalid recommendations. The determinations made independently by the developers for each option are given in Table 1. Some developers, with a small rate of 0.53%, did not express an opinion for a small number of records. According to these findings, when a proportion is established for 3000 reviews, the "Not suitable" option corresponds to a rate of 61.9%. When we look at the majority vote for each record separately, the "not suitable" rate increases to 68%. Blank selections were accepted as "Not suitable" when deciding on each record. As a result of evaluating the relationship between the suggested and expected comments, it was concluded that the suggestion made to 32% of the records was appropriate. Other comments were considered not to provide exactly the expected recommendation. The study conducted here is based on the hypothesis that records scored low by the BLEU score may actually be successful propositions. The opposite may also happen. Based on this, it can be said that manual analyses are still needed in the evaluation of computer-aided predictions.

3.2.8. Model parameters and system specifications

Both TF-IDF and BoW vectorizers were taken from the sklearn [40] library, and they were used with their default parameters; no other configuration was applied. The Word2Vec and Doc2Vec models were taken from the gensim [41] library. They are trained with parameters shown in Table 2. The pre-trained Code2Vec model is taken from its official github repository,¹ and it is used with its default parameters [42]. Lastly, we used the pre-trained Transformer model, which has already been uploaded to the huggingface [43] by the authors of the UniXCoder [31], and no other fine-tuning or hyperparameter tuning was applied. The vector size output by all models is shown in Table 3.

The system specifications used while performing the experiments are as follows: Intel Core i7-10750H; 16 GB Ram; NVIDIA GTX1650; and Windows 11.

All trained models and all experimental materials are publicly available to reproduce our results.²

¹ <https://github.com/tech-srl/code2vec>

² <https://github.com/ykartal/Github-SourceCode-Review>

Table 1
Developers' findings for manual investigation.

	Suitable	Alternative solution	Same meaning	Not suitable	No comment
Developer 1	0	57	8	235	0
Developer 2	7	104	85	104	0
Developer 3	0	65	3	232	0
Developer 4	107	100	34	59	0
Developer 5	26	85	81	107	1
Developer 6	11	62	13	207	7
Developer 7	0	11	3	285	1
Developer 8	0	9	0	291	0
Developer 9	18	139	78	58	7
Developer 10	0	19	2	279	0
	5.63%	21.7%	10.23%	61.9%	0.53%

Table 2
Word2Vec and Doc2Vec parameters.

Parameter	Doc2Vec value	Word2Vec value
vector_size	300	300
window	15	15
min_count	5	10
epochs	20	5
sg	n/a	1
hs	0	0
dm	0	n/a
sample	0.00001	0.00001
alpha	0.03	0.03
min_alpha	0.0007	0.0007
workers	8	8
negative	5	5

Table 3
Vector sizes.

Model	Vector size
Transformer	768
BoW	376 565
Doc2Vec	300
TF-IDF	376 565
Word2Vec	300
Code2Vec	384

4. Experimental results

The results of all of the experiments were evaluated based on perfect prediction and execution time, as well as the NLG metrics. It should be noted that an actual comparison is made with the BoW-Cosine-Bleu combinations, the model proposed in a previous study [12]. The numbers in the tables show the average results of the folds in the 5-fold cross-validation.

4.1. Vectorizer comparison

The results of the experiments with the six different vectorizer models and different candidate counts (1, 3, 5, 10) are given in Table 4 and Fig. 4. The only variable under comparison is the vectorizer models in these results (Table 4, Fig. 4). The vector distances are calculated with cosine distance, and the metrics BLEU, the exact match ratio, and the execution time in seconds are evaluated. The proposed model's (Transformer) scores are shown underlined and the previously proposed method's scores are shown in italics and, at each step, the best score (lowest for execution time and highest for the others) is shown in bold in Table 4. Table 4 and Fig. 4 show that the best model is the Transformer according to BLEU, as well as being the exact match ratio. On the other hand, the least time-consuming model is Word2Vec, and Code2Vec is the most ineffective according to the other metrics. Regarding candidate count changes, the BLEU scores and exact match ratios increased as the candidate count increased at each step for every vectorizer model. Furthermore, the difference between the

scores decreased, and the scores became almost the same, except for the Transformer. This is because, as can be seen in Table 3, the Transformer exhibits a better average for all candidate numbers than the scores of the other methods, with a certain difference. We can say that the Transformer model is the most accurate for every candidate count. In a detailed comparison with the previously proposed method BoW, the Transformer model outperforms the BoW model in every candidate count and metric. In the BLEU mean comparison, the Transformer shows an increase of about 7% when the scores for all of the candidate counts are averaged. The Transformer can undoubtedly find comments that match real review comments, averaging about 18% more than the BoW. From a time efficiency perspective, the Transformer works almost 36% faster than the BoW. As a result, when traditional models are grouped as TF-IDF and BoW and word embedding models are grouped as W2V and D2V, it can be seen that the performance of the same group members is closer to each other. The performance of the group members is closer to each other. Word embeddings generally show the least successful results except for time efficiency, and the traditional models take second place. The Transformer performs better than others in terms of suggesting more accurate reviews. In general, the results are presented as the mean value of all folds in a 5-fold cross-validation. Data in Table 8 shows the BLEU mean variance of folds of results in Table 4, and it revealed that the variability amongst the folds was minimal, indicating that our findings are dependable. Each fold could be considered an individual code review flow test set, implying that five distinct data sets were employed to derive each result. These findings provide insight into the robustness and accuracy of our methodology and the effectiveness of our experimental design. The comprehensive evaluation of our approach reinforces the validity of our results.

Table 12 depicts a real-life example of a code review recommendation model, employing diverse vectorization techniques and the cosine similarity approach with ten candidate counts. The results indicate that, in the given scenario, the recommendations provided by the Transformer method are the same as the actual review comment. This table also presents the BLEU scores of the recommendations provided by other methods.

4.2. Comparison of different combinations

Three NLG Metrics (BLEU, Meteor, Rouge-L) and three vector distance methods (Cosine, Euclidean, Manhattan) are evaluated with five vectorization models to code review automation. The results are measured for the best ten candidates in every combination, and the highest NLG metric score is taken from these ten candidates. These scores are shown in Tables 5–7 for the NLG Metric Mean, Exact Match Ratio, and Execution Time in seconds. We mainly take BLEU for our proposition, but we also use METEOR and ROUGE-L to illustrate how these models perform for them and to pave the way for different comparisons in future research. The cosine distance method has the highest preference rate for subtasks that depend on vector comparisons. We also use Euclidean and Manhattan for this evaluation to display how they work on NLG metrics and vectorization models.

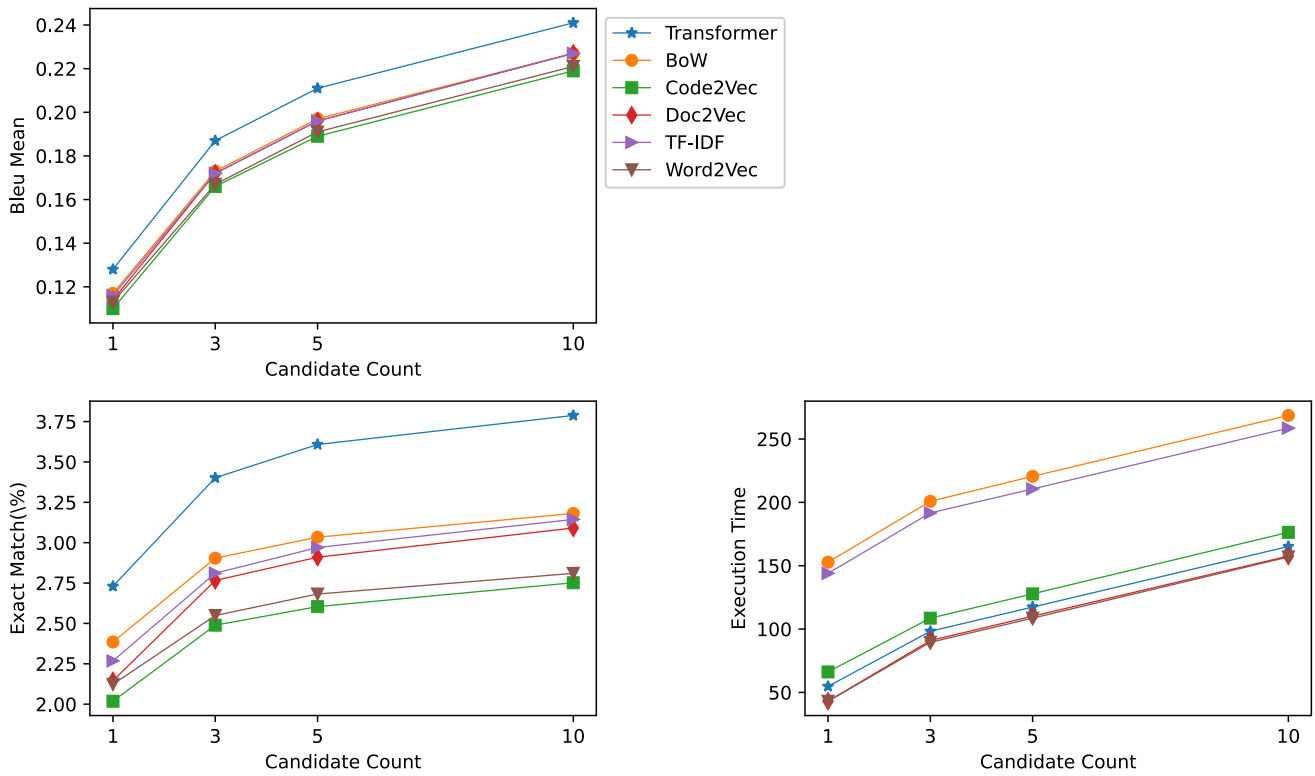


Fig. 4. Comparison of methods with different candidate count.

Table 4

Evaluation of the comparison between vectorization methods.

Candidate count	Bleu mean				Exact match (%)				Execution time (s)			
	1	3	5	10	1	3	5	10	1	3	5	10
Transformer	0.127	0.185	0.210	0.241	2.799	3.381	3.638	3.788	59.171	103.569	171.812	163.048
BoW	0.117	0.173	0.197	0.227	2.385	2.904	3.034	3.181	129.201	176.234	195.284	254.778
Doc2Vec	0.114	0.172	0.196	0.227	2.145	2.766	2.910	3.091	43.269	94.470	111.282	161.107
TF-IDF	0.116	0.172	0.196	0.227	2.268	2.811	2.970	3.144	125.494	174.118	176.571	241.050
Word2Vec	0.113	0.167	0.191	0.221	2.126	2.552	2.683	2.808	38.912	85.153	105.513	145.805
Code2Vec	0.110	0.166	0.189	0.219	2.018	2.489	2.604	2.752	66.175	108.608	127.805	176.354

In Tables 5–7, the main comparison is between the BoW and the Transformer methods. The other methods show the experiments' results and give examples of how the results are obtained with different methods. Looking at the exact match and the NLG metric mean results, the Transformer was more successful in every combination. Compared with the BoW, the Cosine-Transformer pair shows a 6.2% increase in the average BLEU score, a 19.1% increase in the exact match rate, and a 36% acceleration in runtime. General comparisons are given below.

In every measured NLG metric mean, the Transformer model shows the best performance. In cosine distance, the BoW, Doc2Vec, and TF-IDF give almost identical results in every NLG metric, and Word2Vec gives the worst results. In Euclidean distance, there are slight differences between the vectorizers' performance, excluding the Transformer, but generally, the TF-IDF is the second best. For different NLG metrics, performances change slightly. In the Manhattan distance, the results are similar to the Euclidean's. Consequently, the highest performance is precisely taken from the Cosine distance in every NLG metric. The Transformer model gives the highest accurate predictions in every combination. The results vary in vector distances, in the order of the best to the worst Cosine, Euclidean, and Manhattan. The BoW, TF-IDF, and Doc2Vec performances are the highest in cosine, but the Word2Vec works better with the Euclidean. In every vector distance measurement, the highest value is taken from BLEU. As a result, it can be seen that the highest BLEU value in each vector distance measurement was obtained with the Cosine-Transformer pair (Table 5).

Table 5

Mean NLG metric scores of applied methods.

		Transformer	BoW	Doc2Vec	TF-IDF	Word2Vec	Code2Vec
C	Bleu	0.241	0.227	0.227	0.227	0.221	0.219
	Meteor	0.177	0.165	0.166	0.165	0.160	0.158
	Rouge	0.266	0.254	0.253	0.253	0.249	0.246
E	Bleu	0.239	0.220	0.217	0.225	0.221	0.214
	Meteor	0.176	0.160	0.158	0.163	0.159	0.155
	Rouge	0.260	0.248	0.242	0.250	0.249	0.243
M	Bleu	0.239	0.220	0.216	0.224	0.220	0.215
	Meteor	0.176	0.161	0.158	0.162	0.159	0.155
	Rouge	0.260	0.248	0.242	0.248	0.249	0.244

C: Cosine, E: Euclidean, M: Manhattan.

It is seen that the best pair for exact match evaluation is the Cosine-Transformer (Table 7). In execution time evaluation, Cosine is the most efficient, and Manhattan is the least efficient. In particular, Manhattan is extremely slow regarding Transformer and word embedding models. When we look at the results production speeds of the metrics, Rouge is the fastest, and Meteor is the slowest (Table 6).

In this study, the outcomes are presented as the mean value of the folds in a 5-fold cross-validation technique. Data in Table 8 shows the variance of folds of results in Table 4, and it revealed that the

Table 6

Execution time of applied methods (s).

		Transformer	BoW	Doc2Vec	TF-IDF	Word2Vec	Code2Vec
C	Bleu	163.05	254.78	161.11	241.05	145.80	176.35
	Meteor	267.53	357.10	260.12	335.74	257.73	300.91
	Rouge	148.98	215.83	144.33	249.27	139.81	177.28
E	Bleu	234.05	260.55	193.60	243.19	206.92	178.21
	Meteor	310.05	373.27	312.05	354.61	314.20	302.57
	Rouge	199.61	257.41	179.05	237.32	199.69	176.34
M	Bleu	1942.33	284.08	1010.25	258.21	1021.55	1131.83
	Meteor	1921.62	388.21	1081.34	362.01	1098.42	1259.88
	Rouge	1850.43	269.47	971.73	248.72	984.03	1125.74

C: Cosine, E: Euclidean, M: Manhattan.

Table 7

Exact match percentage rate of applied methods.

		Transformer	BoW	Doc2Vec	TF-IDF	Word2Vec	Code2Vec
C	Bleu	3.788	3.181	3.091	3.144	2.808	2.752
	Meteor	3.769	3.170	3.084	3.133	2.797	2.736
	Rouge	3.711	3.117	3.030	3.087	2.758	2.700
E	Bleu	3.701	2.802	2.595	2.992	2.825	2.520
	Meteor	3.687	2.793	2.589	2.981	2.817	2.510
	Rouge	3.620	2.754	2.549	2.937	2.783	2.478
M	Bleu	3.696	2.929	2.562	2.947	2.807	2.560
	Meteor	3.683	2.919	2.552	2.940	2.798	2.548
	Rouge	3.618	2.875	2.516	2.906	2.755	2.508

C: Cosine, E: Euclidean, M: Manhattan.

Table 8

Fold variances of BLEU means in Table 4.

Candidate count	Variance			
	1	3	5	10
Transformer	1.20e-06	2.02e-06	1.69e-06	1.67e-06
BoW	3.14e-06	4.23e-06	3.26e-06	1.84e-06
Doc2Vec	1.48e-06	1.92e-06	1.72e-06	1.66e-06
TF-IDF	1.97e-06	1.61e-06	1.98e-06	1.69e-06
Word2Vec	9.10e-07	1.87e-06	1.87e-06	2.18e-06
Code2Vec	6.44e-06	4.58e-03	8.87e-07	1.33e-06

Table 9

Normal distribution of 5-fold BLEU values (k = 10).

Method	Skewness	Kurtosis	Mean	Median
Transformer	-1.004 ^a	0.66 ^a	0.2410	0.2411
BoW	0.062 ^a	0.971 ^a	0.2268	0.2271
Doc2Vec	1.988	4.123	0.2265	0.2263
TF-IDF	-0.827 ^a	-0.039 ^a	0.2271	0.2272
Word2Vec	-0.423 ^a	-0.963 ^a	0.2265	0.2263
Code2Vec	-0.451 ^a	-1.299 ^a	0.2187	0.2189

^a -1.5<stat<1.5: Normal distribution is provided.

variability amongst the folds was minimal, indicating that our findings were dependable. Each fold could be considered an individual code review flow test set, implying that five distinct data sets were employed to derive each result. According to the data obtained from Table 9, the results are normally distributed. For this reason, one-way anova test was applied to see homogeneity and significant difference. Looking at the homogeneity test (Table 10), it is seen that the averages are distributed homogeneously ($p > 0.05$). According to the one-way anova test (Table 11), there is a significant difference between the Transformer and the other applied methods ($p < 0.05$).

These findings highlight the robustness and accuracy of our methodology and provide insight into the effectiveness of our experimental design. The comprehensive evaluation of our approach reinforces the validity of our results.

Table 10

Tests of homogeneity of variances (k = 10).

	Levene statistics	p (significance)
Based on Mean	0.405	0.840 ^a
Based on Median	0.407	0.839 ^a

^a $p > 0.05$: The homogeneity assumption of the variance is met.**Table 11**

Bonferroni test results for comparison between transformer and other methods (k = 10).

	Mean difference	Std. error	p
BoW	0.0142	0,000840	<0.001 ^a
Doc2Vec	0,0145	0,000840	<0.001 ^a
TF-IDF	0,0139	0,000840	<0.001 ^a
Word2Vec	0,0145	0,000840	<0.001 ^a
Code2Vec	0,0223	0,000840	<0.001 ^a

^a $p < 0.05$: The mean difference is significant at the 0.05 level.

Table 12 depicts a real-life example of a code review proposal model, employing diverse vectorization techniques and the cosine similarity approach with ten candidate counts. The results indicate that, in the given scenario, the recommendations provided by the Transformer method are the same as the actual review comment. Additionally, Table 12 also presents the BLEU scores of the recommendations provided by other methods.

5. Threats to validity and limitations

In this section, we discuss the limitations and threats of this research. Despite our careful design, potential threats to the validity of our results and conclusions may still exist. In this discussion, we address these threats and their potential impact on our study's outcomes.

Although this study's results are more successful than those of other studies, the developed system suggests similar code comments rather than producing a new code comment. If the model encounters code it has not encountered before, it is highly likely to suggest invalid comments. For this reason, it should be considered that more realistic suggestions can be presented using generative AI in future studies.

On the other hand, BLEU, METEOR, and ROUGE scores alone are insufficient as they are based on the similarity between the recommended and actual comments. As wildly different comments can refer to the same issues, low similarity does not mean the recommendation is wrong. Similar comments may refer to very different issues, so a high similarity does not mean that comments suggest the same. A recommendation may say similar things, but the recommendation is not helpful. For this reason, 300 code suggestions presented by the system we developed were examined and evaluated one by one by ten software developers. However, approximately 150,000 suggestions have still not been manually reviewed. Although the previous studies cleaned the dataset, it may still include trivial things, such as whitespace and coding style.

More than one comment may have been made for change requests on the same code at different times. It is very difficult to detect these in the dataset studied. If there are, these code fragments can appear in the train and test data simultaneously, even though they have the same history. Although the presence of this situation may affect the measurement of success criteria, it can appear before developers in a real scenario and provide valuable reminders about the history of that piece of code.

Internal validity. The software developers who decided to participate in manual investigation freely decided to do so. So there may be possible subjectiveness issues in the manual analyses. We had tried to mitigate this threat by 10 developers inspecting the same instance. Still, as for any manual process, imprecisions are possible.

External validity. The datasets on which the applied methods are trained include Java codes. For this reason, it may not be able to make correct recommendations for other programming languages.

Table 12
Real world example.

Changed code	<pre>public String getDirectoryUrl() { return directoryAsyncClient.getDirectoryUrl(); }</pre>	
		Bleu score
Actual Comment	This should return a URL	–
Transformer	This should return a URL	1
BoW	Should return a boolean	0.6433
Doc2Vec	Wrap this line	0.1268
TF-IDF	Is this a textual description or should this be an enum?	0.1301
Word2Vec	Return queryId	0.2551
Code2Vec	This should be private method	0.3892

6. Conclusion

In addition to the deep-learning-based code review generation systems, the IR-based systems achieve acceptable performance in the case of code review comment suggestions. Moreover, their time efficiency is much better than deep-learning-based systems, especially in training costs. The quality of measuring code similarity directly affects these systems, so the accuracy of recommendation is highly correlated with the suitability of how textual data is converted into vectors.

Among the three methods we applied, unlike in the previous methods, the Transformer showed extremely successful results compared to the others. According to the Bleu evaluation, the Transformer method, which we suggested for exact one-to-one matching, offered 19.1% more successful estimations than the BoW method, which gave the closest and highest result. Similarly, in the mean of correct estimation, the Transformer method provided 6.2% more successful estimations than the previously offered state-of-the-art method. Other evaluation methods similarly show that the Transformer offers more successful results. The fact that the method used for vectorization is wide enough to cover all possible texts improves success rates and is seen as extremely timesaving as there is no need for re-vectorization.

7. Future work

The method we propose provides support for the Modern Code Review functionality. Suggestions can be optimized with human-assisted validations. Building a system that makes highly accurate recommendations can serve as an IDE plug-in that makes suggestions and warnings during software development. In future studies, a model could be created by feeding from the existing code repository to suggest updating with the correct source code instead of commenting on the written code.

The literature has many approaches to making suggestions on digital data, so many machine learning methods could be experimented with for better results. According to most of the studies on this domain, the Natural Language Processing (NLP) methods should be the first machine learning methods to test.

The results of this article contribute to the development of systems that will provide an opportunity for developers to be more efficient in modern code review processes. Future studies on improving code similarity measurement with different vectorization techniques besides the Transformer, one of the proposed state-of-the-art vectorization processes, may provide more successful results.

CRedit authorship contribution statement

Yusuf Kartal: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Data curation, Conceptualization. **E. Kaan Akdeniz:** Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Kemal Özkan:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data has been used is published by a previous article and our manuscript has references to that dataset.

References

- [1] Henri Barki, Suzanne Rivard, Jean Talbot, Toward an assessment of software development risk, *J. Manage. Inf. Syst.* 10 (2) (1993) 203–225.
- [2] Ohiduzzaman Shuvo, Parvez Mahbub, Mohammad Masudur Rahman, Recommending code reviews leveraging code changes with structured information retrieval, in: 2023 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2023, pp. 194–206.
- [3] Brian Chess, Gary McGraw, Static analysis for security, *IEEE Secur. Priv.* 2 (6) (2004) 76–79.
- [4] Jevgenija Pantiuchina, Gabriele Bavota, Michele Tufano, Denys Poshyvanyk, Towards just-in-time refactoring recommenders, in: 2018 IEEE/ACM 26th International Conference on Program Comprehension, ICPC, IEEE, 2018, pp. 312–3123.
- [5] Hee Wan Kim, A study on the application of agile methodology to improve software development quality, *Int. J. Adv. Smart Convergence* 9 (3) (2020) 59–70.
- [6] Vipin Balachandran, Reducing human effort and improving quality in peer code reviews using automatic static analysis and reviewer recommendation, in: 2013 35th International Conference on Software Engineering, ICSE, IEEE, 2013, pp. 931–940.
- [7] Devarshi Singh, Varun Ramachandra Sekar, Kathryn T Stolee, Brittany Johnson, Evaluating how static analysis tools can reduce code review effort, in: 2017 IEEE Symposium on Visual Languages and Human-Centric Computing, (VL/HCC), IEEE, 2017, pp. 101–105.
- [8] Atacilio Cunha, Tayana Conte, Bruno Gadelha, Code review is just reviewing code? A qualitative study with practitioners in industry, in: Brazilian Symposium on Software Engineering, 2021, pp. 269–274.
- [9] Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, Alberto Bacchelli, Modern code review: A case study at google, in: Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, 2018, pp. 181–190.
- [10] Tobias Baum, Olga Liskin, Kai Niklas, Kurt Schneider, A faceted classification scheme for change-based industrial code review processes, in: 2016 IEEE International Conference on Software Quality, Reliability and Security, QRS, IEEE, 2016, pp. 74–85.
- [11] Sumaira Nazir, Nargis Fatima, Suriyati Chuprat, Modern code review benefits-primary findings of a systematic literature review, in: Proceedings of the 3rd International Conference on Software Engineering and Information Management, 2020, pp. 210–215.
- [12] Yang Hong, Chakkrit Tantithamthavorn, Patanamon Thongtanunam, Aldeida Aleti, CommentFinder: A simpler, faster, more accurate code review comments recommendation, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022, pp. 507–519.
- [13] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, Gabriele Bavota, Towards automating code review activities, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering, ICSE, IEEE, 2021, pp. 163–174.

- [14] Jing Kai Siow, Cuiyun Gao, Lingling Fan, Sen Chen, Yang Liu, Core: Automating review recommendation for code changes, in: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, 2020, pp. 284–295.
- [15] Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, Chun Zuo, AUGER: automatically generating review comments with pre-training models, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022, pp. 1009–1021.
- [16] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, Gabriele Bavota, Using pre-trained models to boost code review automation, 2022, arXiv preprint arXiv:2201.06850.
- [17] Sanjay Yadav, Sanyam Shukla, Analysis of k-fold cross-validation over hold-out validation on colossal datasets for quality classification, in: 2016 IEEE 6th International Conference on Advanced Computing, IACC, IEEE, 2016, pp. 78–83.
- [18] Anjan Karmakar, Romain Robbes, What do pre-trained code models know about code? in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2021, pp. 1332–1336.
- [19] Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, Liguang Huang, Bin Luo, SPT-code: sequence-to-sequence pre-training for learning source code representations, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2006–2018.
- [20] Patanamon Thongtanunam, Chanathip Pornprasit, Chakkrit Tantithamthavorn, AutoTransform: Automated code transformation to support modern code review process, in: 2022 IEEE/ACM 44th International Conference on Software Engineering, ICSE, 2022, pp. 237–248.
- [21] Chaoyong Ragkhitwetsagul, Measuring code similarity in large-scaled code corpora, in: 2016 IEEE International Conference on Software Maintenance and Evolution, ICSME, IEEE, 2016, pp. 626–630.
- [22] Anshul Gupta, Neel Sundaresan, Intelligent code reviews using deep learning, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18) Deep Learning Day, 2018.
- [23] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, et al., Automating code review activities by large-scale pre-training, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022, pp. 1035–1047.
- [24] Rosalia Tufano, Ozren Dabić, Antonio Mastropaolo, Matteo Ciniselli, Gabriele Bavota, Code review automation: Strengths and weaknesses of the state of the art, IEEE Trans. Softw. Eng. (2024).
- [25] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, Chun Zuo, Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning, in: 2023 IEEE 34th International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2023, pp. 647–658.
- [26] Jaydeb Sarker, Asif Kamal Turzo, Ming Dong, Amiangshu Bosu, Automated identification of toxic code reviews using ToxiCR, ACM Trans. Softw. Eng. Methodol. (2023) in press.
- [27] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint arXiv:2002.08155.
- [28] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al., Codexglue: A machine learning benchmark dataset for code understanding and generation, 2021, arXiv preprint arXiv:2102.04664.
- [29] Zimin Chen, Martin Monperrus, A literature study of embeddings on source code, 2019, arXiv preprint arXiv:1904.03061.
- [30] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, Kensen Shi, Learning and evaluating contextual embedding of source code, in: International Conference on Machine Learning, PMLR, 2020, pp. 5110–5121.
- [31] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, Jian Yin, UniXcoder: Unified cross-modal pre-training for code representation, 2022, arXiv preprint arXiv:2203.03850.
- [32] Masum Hasan, Tanveer Muttaqueen, Abdullah Al Ishtiaq, Kazi Sajeed Mehrab, Md Haque, Mahim Anjum, Tahmid Hasan, Wasi Uddin Ahmad, Anindya Iqbal, Rifat Shahriyar, Codesc: A large code-description parallel dataset, 2021, arXiv preprint arXiv:2105.14220.
- [33] Andriy Mnih, Koray Kavukcuoglu, Learning word embeddings efficiently with noise-contrastive estimation, in: Advances in Neural Information Processing Systems, vol. 26, 2013.
- [34] Mohammed Alsuhailani, Danushka Bollegala, Takanori Maehara, Ken-ichi Kawarabayashi, Jointly learning word embeddings using a corpus and a knowledge base, PLoS One 13 (3) (2018) e0193094.
- [35] Jeffrey Pennington, Richard Socher, Christopher D. Manning, Glove: Global vectors for word representation, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP, 2014, pp. 1532–1543.
- [36] Kishore Papineni, Salim Roukos, Todd Ward, Wei-Jing Zhu, Bleu: A method for automatic evaluation of machine translation, in: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, 2002, pp. 311–318.
- [37] Satandeep Banerjee, Alon Lavie, METEOR: An automatic metric for MT evaluation with improved correlation with human judgments, in: Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization, 2005, pp. 65–72.
- [38] Chin-Yew Lin, Rouge: A package for automatic evaluation of summaries, in: Text Summarization Branches Out, 2004, pp. 74–81.
- [39] Chris Thunes, javalang: Pure python java parser and tools, 2022, <https://github.com/c2nes/javalang>,
- [40] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al., Scikit-learn: Machine learning in python, J. Mach. Learn. Res. 12 (2011) 2825–2830.
- [41] Radim Rehurek, Petr Sojka, Gensim-Python Framework for Vector Space Modelling, vol. 3, (2) NLP Centre, Faculty of Informatics, Masaryk University, Brno, Czech Republic, 2011, p. 2.
- [42] Uri Alon, Meital Zilberstein, Omer Levy, Eran Yahav, Code2Vec: Learning distributed representations of code, in: Proc. ACM Program. Lang., vol. 3, (POPL) ACM, New York, NY, USA, 2019, pp. 40:1–40:29.
- [43] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al., Transformers: State-of-the-art natural language processing, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2020, pp. 38–45.



Yusuf Kartal received his B.S. degree in Computer Engineering from Hacettepe University, Türkiye, in 2007. He worked as a software developer, team leader, and project manager in the defense industry and health fields for over ten years. He received his M.S. degree in Computer Engineering from Anadolu University, Türkiye, in 2017. His research interest includes DevOps and cyber security, especially software security. He is still the founding partner of a software company where he is the CTO and continues to work as an assistant professor in the Department of Computer Engineering at Eskişehir Osmangazi University.



E. Kaan Akdeniz received his bachelor's degree in Computer Engineering from Eskişehir Osmangazi University in 2021. He is currently pursuing his master's degree in Computer Engineering at Izmir Institute of Technology. Since his undergraduate education, he has been working on different areas of artificial intelligence. In particular, he continues his research to specialize in Natural Language Processing.



Kemal Özkan received M.S. and Ph.D. degrees from Eskişehir Osmangazi University in 2000 and 2006. In addition to many administrative duties, he was the head of the computer engineering department at Eskişehir Osmangazi University from 2018 to 2021. He is currently working as a professor at the Department of Computer Engineering at Eskişehir Osmangazi University. His research interests generally cover image and video processing, machine and deep learning, artificial intelligence, and recommender system.